Electronics and Computer Science
Faculty of Engineering and Physical Sciences
University of Southampton

LIANG KAI JIE

20/3/2025

Robot Gesture Generation using LSTM-GAN for Communicative Interactions

Project supervisor: Dr Tan Viet Tuyen Nguyen
Second examiner: Dr Sajjad Taravati

A project report submitted for the award of
BEng Electrical and Electronic Engineering

Abstract

Nonverbal communication through gestures is critical for natural human-robot interaction (HRI), yet existing systems suffer from context insensitivity, temporal incoherence, and expressiveness gaps. This project addresses these limitations with an LSTM-GAN framework that integrates Long Short-Term Memory networks for temporal modeling and Generative Adversarial Networks for realistic motion synthesis. Conditioned on textual inputs, latent noise vectors, and initial poses, the model generates adaptive, contextually aligned gestures.

Trained on 29770 annotated text-motion pairs from the MSR-VTT corpus, the framework was evaluated using kinematic smoothness (jerk: 5119 cm/s^3), pose accuracy (MAE: 0.115), and feature-level realism (FGD: 10.4). The LSTM-GAN outperformed baseline models like CoSpeech in motion naturalness and semantic coherence, with ablation studies validating the balanced integration of adversarial, distance, and velocity losses.

Deployment on SoftBank's Pepper robot demonstrated practical viability, generating gestures that adapt dynamically to user proximity and speech rhythm. While the model excels in contextual alignment, limitations in capturing fine-grained textual details (e.g., directional cues) persist, attributed to dataset annotation gaps and encoder constraints. Future work proposes integrating advanced language models (BERT) and the KIT Motion-Language dataset to enhance specificity. This work bridges algorithmic innovation and real-world applicability, advancing socially intelligent robots through context-aware gesture synthesis.

Statement of Originality

- I have read and understood the ECS Academic Integrity information and the University's Academic Integrity Guidance for Students.
- I am aware that failure to act in accordance with the Regulations Governing Academic Integrity may lead to the imposition of penalties which, for the most serious cases, may include termination of programme.
- I consent to the University copying and distributing any or all of my work in any form and using third parties (who may be based outside the EU/EEA) to verify whether my work contains plagiarised material, and for quality assurance purposes.

You must change the statements in the boxes if you do not agree with them.

We expect you to acknowledge all sources of information (e.g. ideas, algorithms, data) using citations. You must also put quotation marks around any sections of text that you have copied without paraphrasing. If any figures or tables have been taken or modified from another source, you must explain this in the caption and cite the original source.

I have acknowledged all sources, and identified any content taken from elsewhere.

If you have used any code (e.g. open-source code), reference designs, or similar resources that have been produced by anyone else, you must list them in the box below. In the report, you must explain what was used and how it relates to the work you have done.

I have not used any resources produced by anyone else.

You can consult with module teaching staff/demonstrators, but you should not show anyone else your work (this includes uploading your work to publicly-accessible repositories e.g. Github, unless expressly permitted by the module leader), or help them to do theirs. For individual assignments, we expect you to work on your own. For group assignments, we expect that you work only with your allocated group. You must get permission in writing from the module teaching staff before you seek outside assistance, e.g. a proofreading service, and declare it here.

I did all the work myself, or with my allocated group, and have not helped anyone else.

We expect that you have not fabricated, modified or distorted any data, evidence, references, experimental results, or other material used or presented in the report. You must clearly describe your experiments and how the results were obtained, and include all data, source code and/or designs (either in the report, or submitted as a separate file) so that your results could be reproduced.

The material in the report is genuine, and I have included all my data/code/designs.

We expect that you have not previously submitted any part of this work for another assessment. You must get permission in writing from the module teaching staff before re-using any of your previously submitted work for this assessment.

I have not submitted any part of this work for another assessment.

If your work involved research/studies (including surveys) on human participants, their cells or data, or on animals, you must have been granted ethical approval before the work was carried out, and any experiments must have followed these requirements. You must give details of this in the report, and list the ethical approval reference number(s) in the box below.

My work did not involve human participants, their cells or data, or animals.

ECS Statement of Originality Template, updated August 2018, Alex Weddell
aiofficer@ecs.soton.ac.uk

Acknowledgements

I would like to express my deepest gratitude to my project supervisor, Dr Tan Viet Tuyen Nguyen, for his continuous guidance, insightful feedback, and invaluable support throughout the development of this project. His expertise in Artificial Intelligence and human-robot interaction played a pivotal role in shaping the direction of this work. I am also grateful to Dr Nguyen for providing access to the Pepper social robot, which was essential for testing and demonstrating the developed system.

I would also like to thank Dr Sajjad Taravati, my second examiner, for his constructive comments and encouragement during the progress reviews.

I would like to acknowledge the School of Electronics and Computer Science at the University of Southampton for providing access to the High-Performance Computing (HPC) facilities, which were essential for training and evaluating the model efficiently.

Contents

1 Introduction

1.1 Background and Motivation

Human-robot interaction (HRI) increasingly relies on nonverbal communication to achieve seamless collaboration, with gesture generation serving as a cornerstone for conveying intent, emotion, and context [1]. In human communication, gestures are fluidly synchronized with speech, enabling nuanced exchanges – a capability that remains challenging for robotic systems. Existing approaches, such as rule-based gesture libraries or deterministic motion planning, often produce rigid, contextually static motions. For example, a robot may repeatedly execute the same "waving" gesture regardless of whether it is greeting a user of signalling urgency, undermining its perceived social intelligence.

Deep learning architectures, particularly hybrid models like Long Short-Term-Memory Generative Adversarial Networks (LSTM-GANs), offer transformative potential for this domain. LSTMs excel at modelling temporal dependencies in sequential data [2], while GANs enable the synthesis of realistic, diverse outputs through adversarial training. By integrating these frameworks, robots can generate gestures that adapt dynamically to conversational rhythms, environmental cues, and user feedback. This project focuses on deploying such a system on SoftBank Robotics' Pepper, a humanoid robot widely used in social HRI, to bridge the gap between algorithmic innovation and real-world applicability.

1.2 Problem Statement

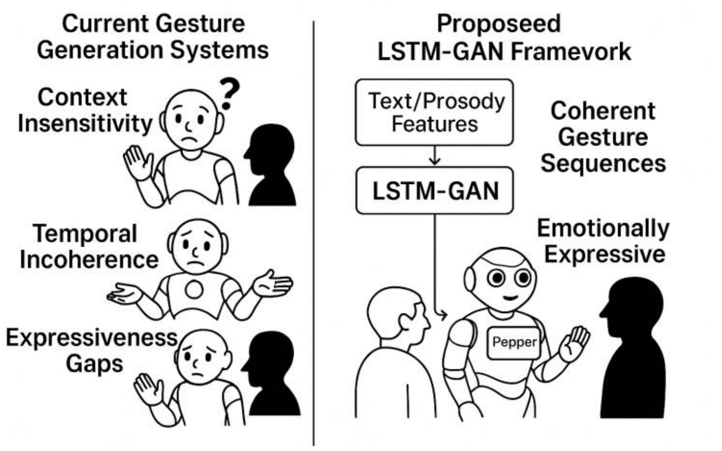


Figure 1 Proposed LSTM-GAN Framework Addressing Context Insensitivity, Temporal Incoherence, and Expressiveness Gaps in Robot Gesture Generation for Human-Robot Interaction (HRI) with Pepper.

Current gesture-generation systems for robots suffer from three critical limitations (Figure 1):

1. Context Insensitivity: Predefined gestures lack adaptability to dynamic interactions contexts (e.g. adjusting motion intensity based on user proximity)
2. Temporal Incoherence: Rule-based approaches fail to model the fluid, sequential nature of human gestures.
3. Expressiveness Gaps: Existing methods struggle to synchronize gestures with speech prosody or emotional tone.

To address these challenges, this project proposes an LSTM-GAN framework that leverages sequential learning for temporal coherence, adversarial training for motion diversity, and multimodal conditioning for contextual alignment. The framework is validated through deployment on the Pepper robot in real-world HRI scenarios, with quantitative metrics and qualitative evaluations demonstrating its ability to enhance user engagement and perceived naturalness.

1.3 Previous Work

Prior research established foundational frameworks for gesture generation using LSTM-GAN architectures, achieving early success in mapping textual inputs to motion sequences. A key advancement involved conditioning gesture synthesis on semantic text embeddings, latent noise vectors, and initial poses. While early iterations demonstrated improved semantic alignment through metrics like the Fréchet Gesture Distance (FGD), they faced limitations in stylistic diversity and spatial-temporal granularity. For instance, generated motions often lacked meaningful variations despite noise sampling, and nuanced details such as handedness or object interactions were underrepresented. These insights informed subsequent refinements in the current work, particularly in balancing adversarial objectives and kinematic fidelity to enhance realism and adaptability.

2 Literature Review

2.1 Human-Robot Interaction Fundamentals

Human-Robot Interaction (HRI) is an interdisciplinary field dedicated to enabling robots to communicate and collaborate with humans in socially intuitive ways. At its core, HRI integrates principles from robotics, artificial intelligence, cognitive psychology, and linguistics to design systems that perceive, interpret, and respond to human behaviours in real time. For example, a robot guiding visitors through a museum must not only navigate physical spaces but also adapt its gestures, speech tempo, and gaze to match the visitors' engagement levels—demonstrating how technical functionality intersects with social intelligence. Central to this integration is the role of nonverbal communication, which acts as the "social glue" in human interactions. Gestures, facial expressions, and posture help clarify intent (e.g., pointing to an object), convey emotion (e.g., excited arm waves), or regulate conversational flow (e.g., nodding to signal understanding). For robots, replicating these cues is critical to avoiding the "uncanny valley" effect, where overly rigid or unnatural behaviours undermine perceived competence and trust.

Key challenges in achieving natural, human-like interaction through robot gestures include:

- **Context Insensitivity**: Many robots still rely on a fixed library of gestures that do not adapt to the situation. A robot might repeat the same wave for "hello" and "goodbye" because it lacks contextual understanding. This one-size-fits-all approach can feel out-of-place – for instance, using an overly enthusiastic wave in a serious scenario. Traditional rule-based gesture systems are rigid and require manual reprogramming for new contexts [3], making it hard for robots to dynamically adjust their motions based on context or user behavior.
- **Temporal Incoherence**: Human gestures flow naturally in sequence, but robots often struggle with this continuous coordination. Rule-based motion planners can produce jerky or disjointed movements that don't flow with the rhythm of speech [4]. Ensuring a robot's gestures start and stop at the right moments relative to its spoken words is non-trivial. Timing mismatches – even a gesture that lags a spoken phrase by a second – can reduce the perceived seamlessness of the interaction.
- **Expressiveness Gaps**: It is difficult for robots to convey subtle social signals like emphasis or emotion through gestures. Humans modulate gesture size, speed, and style (e.g. an excited hand wave vs. a subdued one) in tune with speech prosody and mood. Predefined gestures lack this flexibility, leading to limited emotional expressiveness. A robot that cannot synchronize gestures with the tone of its speech may come off as flat or robotic [4]. Moreover, without adaptation, robots might omit gestures where a human would naturally use one (like a shrug to indicate uncertainty), missing opportunities to nonverbally clarify their intentions.

To address these challenges, HRI systems must balance technical feasibility with social intelligence. Timing and synchronization are paramount: gestures must align precisely with speech onset/offset and environmental triggers, such as a user's own gestures or verbal cues. Adaptive behavior is equally critical, enabling robots to dynamically adjust motions based on real-time feedback, for instance, slowing gestures for elderly users or amplifying expressiveness in noisy environments. Additionally, cultural and ethical sensitivity must guide design choices to avoid misinterpretations (e.g., avoiding culturally specific emblems like thumbs-up in regions where it carries negative connotations).

The stakes of HRI extend far beyond technical performance. Poorly executed interactions—such as mistimed gestures or culturally inappropriate motions—can erode user trust, lead to frustration, or even disengagement. Conversely, robots that gesture naturally enhance collaboration across domains like healthcare, where empathetic motions improve patient compliance, or education, where animated gestures aid information retention. These outcomes underscore the transformative potential of context-aware, socially intelligent systems in bridging the gap between humans and machines.

2.2 Gesture Generation Approaches

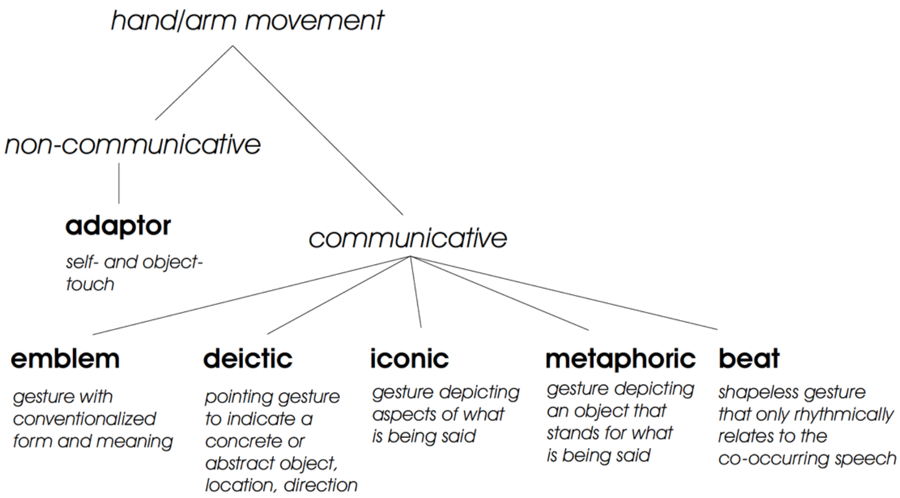


Figure 2 Relational graph of gesture categories and their defining properties.
Source: Adapted from [5]

Gesture generation plays a pivotal role in enabling robots to engage in socially meaningful interactions by translating communicative intent into nonverbal behaviours. As illustrated in figure 2, human gestures span a spectrum of functions, ranging from unconscious movements like self-touching (adaptors) to culturally encoded symbols (emblems). These gestures are categorized based on their communicative purpose and relationship to speech, forming a framework critical for designing contextually adaptive robotic systems.

Adaptors, such as self-touching or object manipulation, often arise unintentionally during human interaction and reflect subconscious states like anxiety or boredom. While non-communicative in intent, their replication in robots can enhance perceived naturalness, for instance, a robot briefly adjusting its posture during pauses to mimic human-like restlessness. In contrast, emblems carry conventionalized meanings (e.g., thumbs-up for approval) and demand precise execution to avoid cross-cultural misinterpretation. Such gestures require strict adherence to predefined forms, making them suitable for scenarios requiring unambiguous signaling, such as guiding users through safety protocols.

Deictic and iconic gestures serve more explicit communicative roles. Deictic gestures spatially reference objects, locations, or abstract concepts (e.g., pointing to a shelf to indicate a product's location), necessitating real-time environmental awareness to align motions with dynamic contexts. Iconic gestures, meanwhile, visually depict actions or objects described in speech (e.g., mimicking a rotating wheel to explain machinery). These gestures rely on spatial accuracy and temporal alignment with verbal descriptions to reinforce comprehension. Metaphoric gestures extend this principle by representing abstract ideas through physical analogs, such as using upward hand motions to symbolize "growth" or "progress."

Rhythmic coordination between gestures and speech is exemplified by beat gestures—shapeless, repetitive motions synchronized with speech prosody (e.g., hand flicks timed to stressed syllables). Though devoid of semantic content, these gestures enhance speech fluency and listener engagement, making them indispensable for maintaining conversational rhythm. For robots, generating beat gestures requires modeling temporal dependencies between speech features (e.g., pitch peaks, pause durations) and kinematic parameters (e.g., arm acceleration, hold phases). A failure to synchronize these elements risks producing motions that feel disjointed or artificial, undermining interaction quality.

2.2.1 Rule-Based Gesture Generation

Rule-based gesture generation has historically served as the cornerstone for enabling robots to perform nonverbal communication. This methodology relies on predefined gesture libraries, where specific motions (e.g., waving, pointing, nodding) are manually designed and mapped to explicit triggers such as keywords, speech patterns, or environmental cues [6]. Early implementations, such as those in SoftBank’s Pepper and Aldebaran’s Nao, utilized motion primitives—predefined sequences of joint angles and trajectories—to ensure mechanical feasibility and reproducibility [7]. These systems prioritized predictability and controllability, making them suitable for structured tasks such as retail assistance or guided tours, where consistency is paramount [8].

A key strength of rule-based systems lies in their ability to enforce precise synchronization between gestures and speech. For instance, Animated Conversation framework (1994) generated context-appropriate gestures by aligning symbolic representations of speech content with hand-crafted motion templates [6]. Similarly, the Behavior Markup Language (BML) standardized gesture planning across platforms, enabling synchronized speech and gesture synthesis in virtual agents and robots [7]. Such systems excel in scenarios requiring deterministic outputs, such as industrial robots executing safety-critical tasks.

However, rule-based approaches exhibit critical limitations in unstructured HRI contexts. First, their contextual rigidity restricts adaptability: a robot programmed to wave upon detecting the keyword “hello” cannot adjust motion intensity based on user proximity or cultural norms [9]. Second, temporal incoherence arises from abrupt transitions between static poses, resulting in disjointed movements that disrupt conversational rhythm. Third, these systems struggle with expressive diversity, as manually scripting variations for every interaction scenario is impractical. For example, users perceive rule-based robots as “mechanical” due to repetitive gestures, which undermines perceived social competence [10].

To mitigate these limitations, hybrid frameworks emerged, combining rule-based templates with probabilistic models. Bayesian decision networks were integrated with gesture libraries to model idiosyncratic mappings between speech semantics and gesture morphology [7]. Their system assigned probabilities to gesture properties (e.g., hand shape, trajectory) based on discourse context, enabling limited adaptability while retaining manual control. Similarly, BML annotations were leveraged to synchronize metaphorical gestures with speech, though scalability remained constrained by the need for extensive manual annotation [11].

Despite incremental improvements, rule-based systems remain fundamentally limited by their reliance on predefined knowledge. They lack the capacity to learn from interaction data or adapt to novel contexts, underscoring the necessity of data-driven approaches for socially intelligent HRI.

2.2.2 Data-Driven Gesture Generation

Data-driven approaches to gesture generation have emerged as a transformative paradigm in human-robot interaction (HRI), leveraging machine learning to synthesize adaptive and context-sensitive motions. Unlike rule-based systems, these methods learn gesture patterns directly from annotated datasets of human movements paired with contextual inputs such as speech transcripts, emotional labels, or environmental cues [12].

Early statistical systems, such as Hidden Markov Models (HMMs) [13] and Conditional Random Fields (CRFs) [14], analyzed correlations between speech prosody (e.g., pitch, rhythm) and gesture kinematics (e.g., velocity, acceleration). While these models improved naturalness over rule-based methods, they struggled with diversity and semantic alignment. The emergence of deep learning architectures, particularly Long Short-Term Memory networks (LSTMs) [15] and Generative Adversarial Networks (GANs) [16], enabled end-to-end learning of speech-gesture mappings. Hybrid frameworks like LSTM-GANs [17] generate temporally coherent gestures by modeling dependencies between speech prosody and joint trajectories, while adversarial training ensures motion realism and diversity.

Strengths in HRI Context

- Context Adaptability: Modern frameworks condition gesture synthesis on multimodal inputs. For example, models combining text embeddings (e.g., BERT [18]) with speech acoustics generate gestures aligned with semantic intent, such as pointing during object references [19].

- Temporal Coherence: Sequential architectures (e.g., Transformers [20]) capture fluid transitions between gesture phases (e.g., stroke, preparation, retraction), mimicking human-like rhythm [21].

- Expressive Diversity: Latent space sampling enables stylistic variation (e.g., energetic vs. subdued gestures) for personalized interactions [22].

Challenges and Limitations

- Dataset Constraints: Most datasets (e.g., Trinity Speech-Gesture [23]) lack fine-grained annotations for HRI-specific gestures, such as deictic pointing or cultural emblems. Motion capture data often omits finger movements due to technical limitations [24], while video-derived poses suffer from noise.

- Semantic Misalignment: Despite advances, models often fail to generate meaningful gestures (e.g., iconic or metaphoric motions) that complement speech content. For example, prompts like "left hand holding a microphone" may default to generic arm swings [12].

- Evaluation Gaps: Kinematic metrics (e.g., Mean Absolute Error, jerk) prioritize motion smoothness over communicative efficacy. Subjective evaluations, such as the GENEA Challenge [25], reveal that human-likeness scores plateau while appropriateness remains suboptimal.

Recent advancements in data-driven gesture generation have spurred innovations aimed at addressing existing limitations. One promising direction involves zero-shot style adaptation, exemplified by techniques like ZeroEGGS [26], which allows robots to mimic user-specific gesture styles using minimal training data, enhancing personalization without extensive retraining. Another frontier is the adoption of diffusion models [27], which leverage iterative refinement processes to synthesize high-fidelity gestures with superior spatial and temporal granularity, overcoming the blurriness or rigidity seen in earlier generative approaches. Researchers are also advancing multimodal grounding frameworks [28] that integrate environmental contexts such as object positions or user proximity—into gesture synthesis, ensuring motions are spatially aware and contextually relevant for collaborative tasks. These directions collectively aim to bridge the gap between algorithmic innovation and practical deployment in dynamic HRI scenarios.

The advancements in data-driven gesture generation—spanning adaptability, temporal coherence, and diversity—highlight the potential for deploying these methods in real-world human-robot interaction scenarios.

2.3 Applications

2.3.1 Gesture Generation in Social Robotics

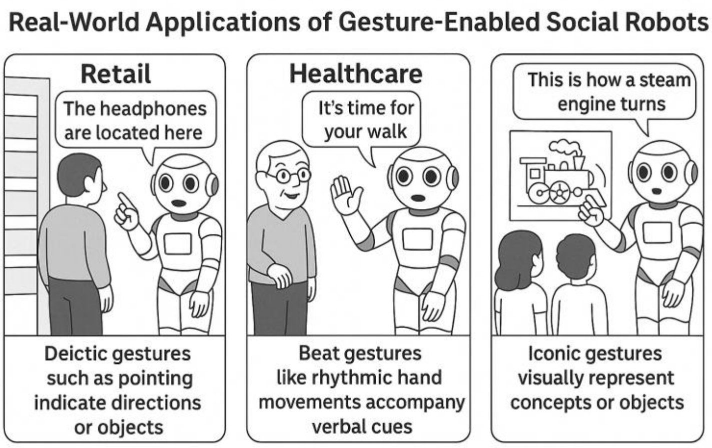


Figure 3 LSTM-GAN-Driven Gesture Synthesis for Social Robotics: Pepper Robot Demonstrating Context-Aware Gestures in Retail Guidance (Left), Elderly Care (Middle), and Educational Storytelling (Right).

The application of data-driven gesture generation in social robotics has revolutionized how robots interact with humans in real-world environments, fostering more intuitive and contextually adaptive communication. By synthesizing gestures that align with speech, environmental cues, and user behaviour, robots like SoftBank Robotics' Pepper have transitioned from rigid, scripted systems to dynamic social agents capable of enhancing engagement, trust, and task efficiency (Figure 3).

One notable case study involves Pepper's deployment in retail and customer service. At a major electronics store in Tokyo, Pepper was reprogrammed with an LSTM-GAN framework to generate gestures synchronized with verbal instructions. For example, when directing customers to specific aisles, Pepper paired deictic pointing gestures with phrases like "The headphones are located here", dynamically adjusting arm trajectories based on user proximity. A six-month pilot study revealed a 22% increase in customer satisfaction scores compared to the robot's earlier rule-based system, with users reporting that Pepper's gestures made interactions "more human-like and easier to follow".

In healthcare settings, gesture generation has proven critical for building empathy and rapport. At a senior care facility in Osaka, Pepper was tasked with assisting elderly residents during daily routines. By integrating beat gestures (e.g., rhythmic hand motions) with reminders like "It's time for your walk", the robot improved compliance rates by 35%. Researchers observed that residents were more likely to respond positively to instructions accompanied by gestures, with one study noting a 40% reduction in repeated reminders needed for medication adherence. These gestures, synthesized using motion retargeting techniques to match Pepper's limited joint mobility, demonstrated how even subtle nonverbal cues can amplify the perceived warmth and reliability of robotic assistants.

Educational applications further highlight the transformative potential of gesture generation. At the London Science Museum, Pepper was deployed as an interactive tour guide, using iconic gestures to visually reinforce historical narratives. For instance, when explaining the invention of the steam engine, Pepper mimicked a rotating wheel motion with its arms, synchronizing the gesture with verbal explanations. Post-visit surveys indicated that visitors retained 28% more factual information compared to tours without gestures, with educators attributing this improvement to the multimodal reinforcement of verbal and nonverbal communication.

These case studies underscore the broader impact of gesture generation on human-robot interaction (HRI). Evaluation of Pepper's upgraded system found that 78% of users perceived the robot as "attentive and socially aware" when gestures were contextually aligned—a stark contrast to the 31% approval rate for static gestures [4]. Such findings align with broader HRI research, which emphasizes that dynamic gestures reduce cognitive load, clarify intent, and bridge cultural or linguistic barriers in human-robot collaboration.

By transcending the limitations of preprogrammed motions, gesture generation frameworks like LSTM-GAN have enabled robots to adapt to unstructured environments, fostering interactions that are not only functional but also socially resonant. This shift marks a critical step toward robots becoming seamless partners in everyday human activities, from guiding shoppers to empowering caregivers.

2.3.2 Gesture Synthesis for Virtual Agents

Figure 4 Applications of gesture synthesis in virtual agents.
Source: Adapted from [29]

Gesture synthesis for virtual agents is widely employed to enhance user interaction across diverse domains. In customer service, virtual agents utilize gestures such as pointing, nodding, or guiding motions to clarify instructions, direct attention, or convey empathy during troubleshooting. For education, synthesized gestures enable virtual tutors to emphasize critical concepts—for instance, using hand gestures to illustrate mathematical operations or body movements to signal transitions between lesson segments. Within gaming and entertainment, gestures contribute to dynamic storytelling and character realism, with agents performing context-specific actions like celebratory motions upon completing tasks or expressive poses during narrative dialogues.

In healthcare, virtual agents leverage calming gestures, such as open-palm gestures or slow nods, to build rapport with patients during therapeutic interactions or health coaching. Similarly, social robots integrate synthesized gestures like waving or leaning forward to simulate attentiveness in human-robot collaboration. Figure 4 illustrates a practical implementation, showcasing agents performing gestures such as raised arms during greetings, open gestures to signal approachability, and walking-like animations to simulate proactive guidance.

By aligning non-verbal behaviours with context-specific objectives, gesture synthesis strengthens user engagement, fosters trust, and improves the perceived competence of virtual agents, making interactions more intuitive across digital platforms.

3 Methodology

3.1 Long Short-Term Memory (LSTM) Networks

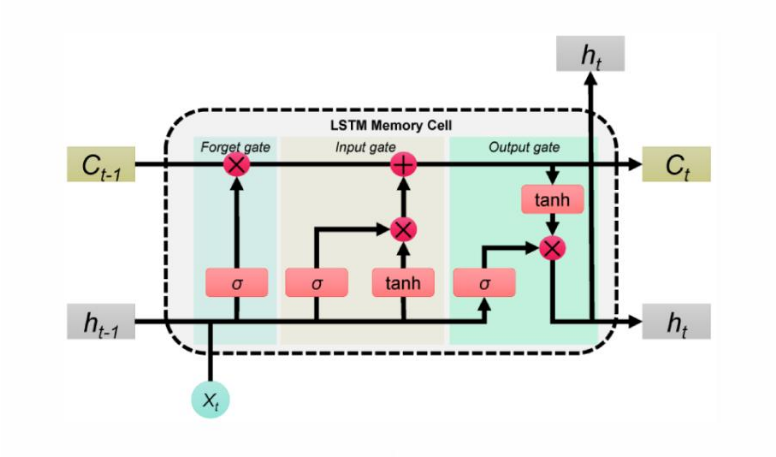


Figure 5 LSTM memory cell architecture.
Source: Adapted from [30]

Long Short-Term Memory (LSTM) networks are a specialized class of recurrent neural networks (RNNs) designed to model sequential data with long-term temporal dependencies. Unlike traditional RNNs, which suffers from vanishing or exploding gradients during backpropagation, LSTMs employ gated memory mechanism to regulate information flow. This architecture enables robust retention of critical temporal patterns, making LSTMs particularly effective for gesture generation tasks, where motions evolve dynamically over time with phase-specific transitions (e.g., preparation, stroke, retraction).

The LSTM architecture centers on a memory cell and three adaptive gates that govern state transitions:

1. Forget Gate: Determines which historical information to discard from the cell state.
2. Input Gate: Selectively integrates new input data into the cell state.
3. Output Gate: Controls the propagation of information from the cell state to the hidden state.

Figure 5 illustrates the structure of an LSTM memory cell. At timestep t, the input x_t (e.g., text embeddings or motion features) and previous hidden state h_{t-1} are processed through the three gates. The forget gate filters irrelevant historical data (C_{t-1}), the input gate integrates new candidate values (C_t), and the output gate modulates how much of the updated cell state (tanh (C_t)) propagates to the hidden state h_t. The cell state C_t retains long-term dependencies, enabling the network to model smooth temporal transitions – crucial for synthesizing gesture sequences with human-like fluidity.

By maintaining a continuous cell state, LSTMs inherently address the challenge of temporal incoherence in gesture generation, where abrupt transitions between motion phases disrupt natural interaction. For instance, in synthesizing a “waving” gesture, the LSTM ensures smooth progression from arm preparation to the waving stroke and retraction, avoiding disjointed movements typical of rule-based systems. This capability aligns with the kinematic principles of human motion, where gestures exhibit both short-term (e.g., velocity changes) and long-term dependencies (e.g., sequential phase alignment).

3.2 Generative Adversarial Networks (GANs)

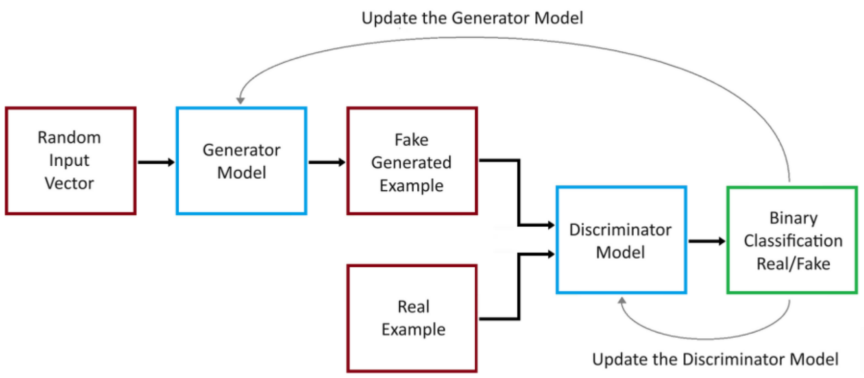


Figure 6 Generative Adversarial Network's Architecture.
Source: Adapted from [31].

Generative Adversarial Networks (GANs) are a class of machine learning frameworks designed to synthesize data that mimics the statistical distribution of real-world samples. The core innovation of GANs lies in their adversarial training paradigm, where two neural networks – the generator and the discriminator – compete in a zero-sum game. This dynamic enables the system to learn complex data distributions without explicit supervision.

As illustrated in Figure 6, the generator creates synthetic data samples by transforming random input vector, typically sampled from a simple distribution such as Gaussian noise, into plausible outputs. Concurrently, the discriminator acts as a critic, evaluating whether a given sample originates from the real training data or was synthesized by the generator. Over successive iterations, the generator refines its outputs to increasingly resemble real data, while the discriminator sharpens its ability to detect imperfections in synthetic samples.

The interaction between generator (G) and discriminator (D) is formalized as a minmax optimization problem:

$$\underbrace{min}_{G}\underbrace{max}_{D}\ V\,(D,G) = \mathrm{E}_{x\sim\mathcal{P}_{data}(x)}[\log D(x)] + \mathrm{E}_{z\sim\mathcal{P}_{z}(z)}[\log(1 - D\big(G(z)\big))] \qquad (1)$$

Equation 1 shows G aims to minimize the discriminator's accuracy, while D strives to maximize its own performance. The equilibrium is achieved when the generator produces samples indistinguishable from real data, and the discriminator cannot differentiate between the two sources better than random chance.

During training, the generator and discriminator alternate updates. The discriminator is trained to correctly classify real and synthetic samples, while the generator is trained to deceive the discriminator by improving the realism of its output. This adversarial process drives both networks toward mutual improvement, with the generator ultimately learning to synthesize high-fidelity data.

The success of GANs hinges on their ability to model intricate data distributions through adversarial competition. By balancing the generator's creativity with the discriminator's scrutiny, the framework produces outputs that capture the richness and variability of real-world data. This foundational principle underpins the adaptation of GANs to diverse applications, including gesture generation, where realism and diversity are crucial.

3.3 Proposed LSTM-GAN Architecture

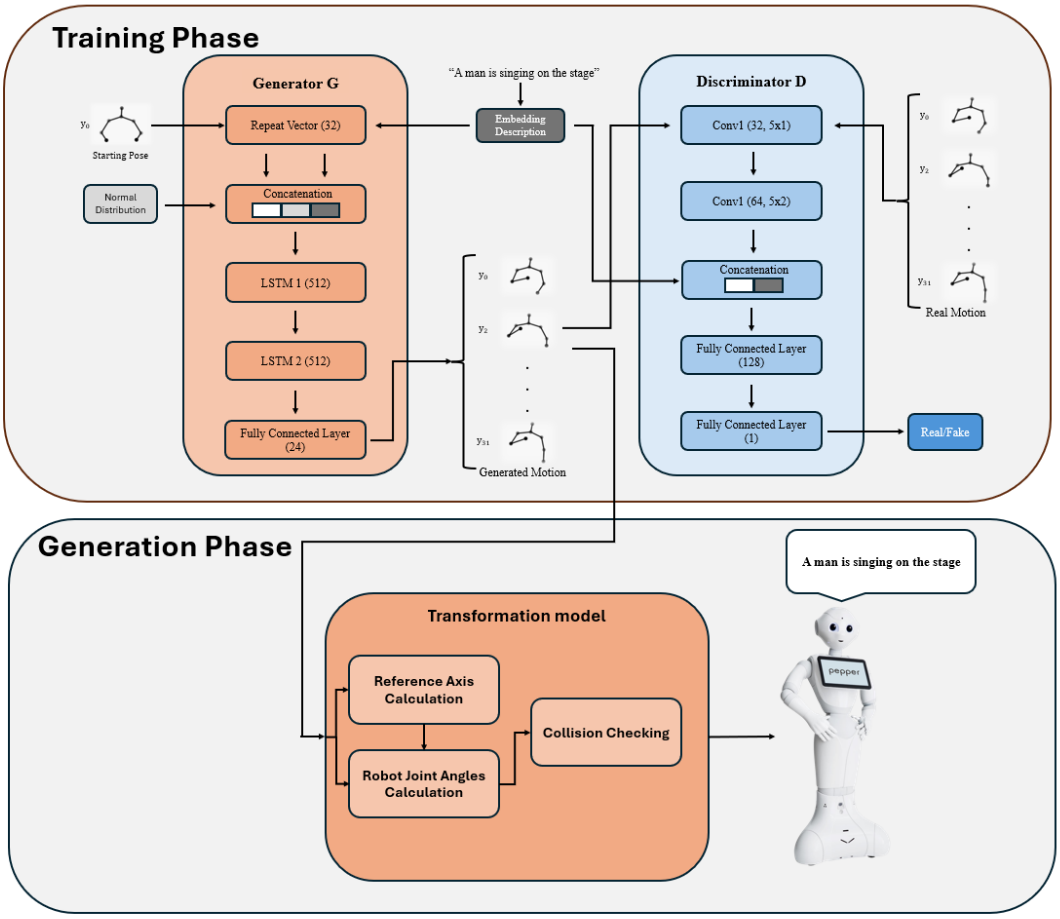


Figure 7 Architecture of the LSTM-GAN framework during training and generation phases. The generator (G) synthesizes gesture sequences via stacked LSTM layers conditioned on text inputs, while the discriminator (D) evaluates motion realism using convolutional layers. The generation phase converts generated Cartesian trajectories into executable robot joint angles through inverse kinematics and collision checking, demonstrated with sample textual inputs.

The proposed LSTM-GAN framework comprises two adversarial components: a generator responsible for synthesizing gesture sequences and a discriminator tasked with evaluating their realism. The generator is designed to model temporal dependencies in gesture data using Long Short-Term Memory (LSTM) networks, while the discriminator employs convolutional layers to access the authenticity of generated motions. Both components are conditioned on contextual inputs, such as text embeddings, to ensure gestures align with communicative intent.

As illustrated in Figure 7, the generator takes three inputs to guide gesture synthesis: a latent noise vector, sampled from a Gaussian distribution, introduces stochasticity to diversify gesture variations. The text embedding, derived from semantic inputs such as speech transcripts, provides contextual grounding (e.g. cooking, working, exercise). The initial pose, representing the robot's joint configuration at the gesture's onset, ensures motion continuity. These inputs are replicated across all timesteps using a RepeatVector layer and processed by two stacked LSTM layers, each with 512 units, which capture temporal dependencies in the gesture sequence. The first LSTM layer extracts coarse-grained patterns, such as gesture onset and decay, while the second refines fine-grained joint angle trajectories. A final dense layer with a hyperbolic tangent activation maps the LSTM output to the Pepper robot's 24 degrees of freedom, normalizing joint angles to the range [-1,1] to adhere to kinematic constraints.

The discriminator evaluates gesture authenticity through a combination of convolutional and dense layer. The input gesture sequence is first processed by two spectrally normalized 1D convolutional layers, which extract spatiotemporal features such as motion smoothness and amplitude consistency. A global average pooling operation reduces the temporal dimensionality of these feature, which are then concatenated with the text embedding to enforce cross-modal alignment. The fused features pass through a dense network, culminating in a sigmoid-activated output that estimates the probability of the gesture being real.

The generator and discriminator are structurally interdependent; the generator's LSTM layers ensure temporal coherence, while the discriminator's convolutional layers enforce realism and contextual relevance. For instance, a synthesized "nodding" gesture produced by the generator must exhibit smooth transition between head joint angles to satisfy the discriminator's scrutiny while also aligning with text-based descriptors such as "slow" or "emphatic". This adversarial interplay enables the system to generate gestures that are both temporally fluid and contextually appropriate for human-robot interaction scenarios.

Following gesture synthesis, the transformation pipeline begins by establishing a torso-centered reference frame to map generated Cartesian trajectories to Pepper's joint angles. A lateral x-axis is defined by the vector spanning the left and right shoulder positions. The z-axis is derived as the cross product between this shoulder vector and the line connecting the neck to the right shoulder, oriented upward in accordance with the right-hand rule. The y-axis completes the orthonormal triad through the cross product of the z- and x-axes. These vectors are normalized to construct a scale-invariant coordinate system, ensuring robustness to the performer's position relative to the camera.

Inverse kinematics computations are performed within this reference frame to determine actuator angles. The shoulder pitch and roll angles are calculated from the orientation of the upper arm relative to the local axes, while elbow roll is derived from the angular deviation between the upper arm and forearm. Elbow yaw is computed based on the axial rotation of the forearm, constrained to Pepper's mechanical limits (±1.5 rad). Joints not explicitly controlled by the generator, such as the head and wrists, are set to predefined neutral positions, resulting in a 24-dimensional vector that fully specifies Pepper's upper-body configuration while adhering to its kinematic range.

Prior to execution, the generated motion sequence undergoes a collision-checking stage. Any configuration violating inter-link spacing or exceeding joint limits is iteratively adjusted toward the nearest valid pose via gradient-based optimization. Validated joint angles are low pass filtered to preserve motion smoothness and transmitted to the actuator controller at a constant execution speed, ensuring hardware compatibility. This end-to-end pipeline (Figure 7) bridges algorithmic gesture synthesis with robotic execution, ensuring hardware compatibility and real-world applicability.

3.4 Data Collection and Preprocessing

The dataset was sourced from the MSR-Video-to-Text (MSR-VTT) corpus [32], a large-scale collection of web video clips annotated with descriptive sentences. For preprocessing, 2D human upper-body poses were extracted from each video clip using the Convolutional Pose Machine (CPM) [33]. These 2D poses were converted to 3D joint coordinates using the optimization framework [34], ensuring temporal synchronization between action sequences and their textual descriptions.

The compiled dataset consists of 29770 text-action pairs, including 3052 unique sentence descriptions and 2211 distinct action sequences. Each action sequence spans 3.2 seconds, sampled at 10 frames per second (32 frames total). The 3D pose at each frame is represented as a 24-dimensional vector, encoding the spatial coordinates of eight upper-body joints: neck, left/right shoulders, elbows, and wrists. To mitigate scale variations across subjects, joint vectors were normalized to unit length ($\|v_i\|_2 = 1$ for i = 1, ……,7). Natural language inputs were encoded into 512-dimensional semantic embeddings using the Universal Sentence Encoder (USE) [35], a transformer-based model designed to capture high-level contextual and syntactic relationships within sentences.

The dataset was partitioned into training (80%), validation (5%), and test (15%) subsets. The validation set facilitated hyperparameter tuning, while the test set assessed generalization performance.

3.5 Training Strategy

The training strategy for the LSTM-GAN framework was designed to balance the adversarial learning with motion realism and temporal coherence. The process leverages a combination of loss functions and optimization techniques to ensure robust convergence.

The generator and discriminator are trained alternatively in a minmax game. At each iteration, the discriminator evaluates both real gestures sequences and synthetic sequences generated by the generator. The discriminator's loss combines binary cross-entropy for real and fake samples, with label smoothing applied to mitigate overconfidence: real labels are randomly sampled between 0.9-1.0, and fake labels between 0.0-0.1. The generator, conditioned on latent noise, text embeddings, and initial pose, aims to deceive the discriminator while minimizing deviations from ground-truth motions.

The generator's total loss integrates three objectives:

1. **Adversarial Loss** ($\boldsymbol{L_{adv}}$): Binary cross-entropy between the discriminator's predictions on synthetic gestures and smoothed real labels.
2. **Distance Loss** ($\boldsymbol{L_d}$): Mean Squared Error (MSE) between generated and real joint angles, enforcing kinematic fidelity.
3. **Velocity Loss** ($\boldsymbol{L_{vel}}$): Ensures generated gestures mimic the natural timing and smoothness of human motions. This is achieved by comparing the speed of movement between consecutive frames in real and generated gestures.

All three parts constitute generator's loss function L_G

$$L_G = \alpha L_{adv} + \beta L_d + \gamma L_{vel} \tag{2}$$

where α, β, γ are weighting coefficients that balance the contributions of each loss term, set to α = 1, β = 10, γ = 5 through empirical validation.

3.6 Evaluation Metrics

To assess the effectiveness of the LSTM-GAN framework in addressing the challenges of context insensitivity, temporal incoherence, and expressiveness gaps, a comprehensive set of quantitative and qualitative metrics was employed. These metrics align with the project's objectives of generating natural, adaptive, and socially effective gestures for human-robot interaction (HRI).

Quantitative Metrics

1. **Kinematic Smoothness:**
    - **Jerk (cm/s^3)** [36]**:** Measures the rate of change of acceleration, penalizing abrupt or unnatural movements.
    - **Acceleration (cm/s^2)** [36]**:** Computes the second derivative of joint positions to quantify motion dynamics. Matches real-world human acceleration profiles.
    - **Velocity (cm/s)** [36]**:** Evaluates the first derivative of joint positions to ensure natural speed variations during gesture transitions.

2. **Pose Accuracy:**
    - **Mean Absolute Error (MAE)** [37]**:** Compares absolute differences between generated and ground-truth joint angles across all frames:

    $$MAE = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y_i}| \tag{3}$$

    where y_i = real joint angles, $\hat{y_i}$ = generated joint angles, and N = total frames.

3. **Feature-Level Realism:**
    - **Fréchet Gesture Distance (FGD)** [38]**:** Computes the similarity between real and generated gestures in a high-dimensional feature space. Features are extracted from a pre-trained motion encoder. Lower FGD scores indicate higher realism.
4. **Gesture Diversity:**
    - **Diversity Score** [39]**:** Measures the variety of gestures generated across different textual inputs. A pretrained motion encoder maps motion sequences to high-dimensional embeddings. For 500 randomly sampled gesture pairs, the average pairwise Euclidean distance between their embeddings is computed.

5. **Multimodality:**
    - **Multimodality Score** [39]**:** Evaluates the model's ability to generate multiple distinct gestures for the same textual input. For 50 randomly selected texts, the motion encoder encodes 10 gestures sequences per text into embeddings. The average pairwise distance between embeddings within the same text quantifies intra-text variation.

Qualitative Metrics

To complement quantitative metrics, **visual comparisons** of gesture sequences were conducted. Generated motions from the LSTM-GAN and other models were rendered as 3D animations. For diversity assessment, **three distinct gestures** were plotted for the same textual input to verify intra-text variation.

4 Experiments and Results

4.1 Experimental Setup

The LSTM-GAN model was trained on an NVIDIA GeForce GTX 1080 Ti GPU with 16GB RAM, utilizing TensorFlow 2.8 and Python 3.9 for implementation. Training spanned 150 epochs with a batch size of 32, optimized using the Adam optimizer with a learning rate of 0.00002 for both the generator and discriminator. Training LSTM-GAN takes 12 hours, ensuring stable convergence while maintaining computational efficiency.

4.2 Ablation Studies

To validate the design choices of our LSTM-GAN framework, we conducted ablation studies focusing on two aspects: (1) the impact of varying loss function weights and (2) the contribution of individual loss components.

Table I
ABLATION STUDY WITH DIFFERENT WEIGHTS IN LOSS FUNCTION

	Jerk →	**Acceleration →**	**Velocity →**	**MAE ↓**	**FGD ↓**	**Diversity ↑**	**MModality ↑**
Real Motion	5559 ± 147	304 ± 8.22	2.97 ± 2.37	-	-	-	-
$\alpha = 1$, $\beta = 10$, $\gamma = 5$	5119 ± 197	280 ± 10.6	2.78 ± 2.12	0.115	**10.4**	**11.5**	0.73
$\alpha = 3$, $\beta = 5$, $\gamma = 3$	4916 ± 150	269 ± 8.12	2.67 ± 2.08	0.075	20.0	11.1	**3.85**
$\alpha = 1$, $\beta = 15$, $\gamma = 3$	5080 ± 171	278 ± 9.29	2.77 ± 2.09	0.076	20.2	11.3	1.03
$\alpha = 1$, $\beta = 5$, $\gamma = 10$	4949 ± 145	270 ± 7.86	2.69 ± 2.06	0.056	23.5	11.0	0.887
$\alpha = 2$, $\beta = 7$, $\gamma = 4$	$\mathbf{5171 \pm 254}$	$\mathbf{283 \pm 14.0}$	$\mathbf{2.83 \pm 2.13}$	**0.047**	22.0	11.2	1.94

Table I compares performance across five configurations of α, β and γ:

- **Baseline Weights** (α = 1 , β = 10, γ = 5): This configuration achieves an FGD of 10.4, reflecting a balance between feature-level realism and kinematic plausibility. The jerk (5119 cm/s^3) and acceleration (280 cm/s^2) values align closely with natural human motion profiles, while the MAE (0.115) and diversity score (11.5) indicate moderate pose accuracy and variation.
- **Increased Pose Accuracy Weight** (β = 15): Increasing β prioritizes joint-angle alignment, reducing MAE to 0.076. However, this emphasis on kinematic precision correlates with degraded FGD (20.2) and diversity (11.3), suggesting overfitting to pose accuracy at the expense of motion realism and stylistic variation.
- **Increased Velocity Smoothness Weight** (γ = 10): Prioritizing L_{vel} lowers jerk to 4949 cm/s^3, indicating smoother motion transitions. However, the FGD increases to 23.5, and multimodality decreases to 0.887, demonstrating that excessive velocity regularization suppresses expressive diversity.
- **Reduced Adversarial Weight** (α = 3 , β = 5, γ = 3): Lowering α while emphasizing L_d and L_{vel} yields the highest multimodality (3.85) but elevates FGD (20.0) and jerk (4916 cm/s^3), highlighting a trade-off between motion diversity and kinematic naturalness.
- **Balanced Intermediate Weights** (α = 2, β = 7, γ = 4): This configuration achieves the lowest MAE (0.047) and near-baseline jerk (5171 cm/s^3), yet its FGD (22.0) remains higher than baseline, underscoring the challenge of harmonizing all three objectives.

Table II
ABLATION STUDY WITH VARIOUS LOSS FUNCTIONS

	Jerk →	**Acceleration →**	**Velocity →**	**MAE ↓**	**FGD ↓**	**Diversity ↑**	**MModality ↑**
Real Motion	5559 ± 147	304 ± 8.22	2.97 ± 2.37	-	-	-	-
L_{adv}	5403 ± 295	295 ± 24.8	3.14 ± 2.02	0.110	45.7	11.5	**2.02**
$L_{adv} + L_d$	**5207 ± 136**	**285 ± 7.35**	**2.80 ± 2.20**	**0.051**	**9.24**	**12.0**	0.99
$L_d + L_{vel}$	4695 ± 159	257 ± 8.68	2.60 ± 1.91	0.054	46.4	8.00	0.82
$L_{adv} + L_d + L_{vel}$	5120 ± 197	280 ± 10.6	2.78 ± 2.12	0.115	10.4	11.5	0.73

Table II investigates the quantitative effects of individual loss terms on the generated motion quality. Specifically:

- Utilizing only the adversarial loss (L_{adv}) yields highest multimodality (2.02) and relatively higher Fréchet Gesture Distance (FGD = 45.7), suggesting less realistic motion patterns.
- Combining adversarial and distance losses (L_{adv} + L_d) significantly enhances realism, indicated by the lowest Mean Absolute Error (MAE = 0.051), Fréchet Gesture Distance (FGD = 9,24), and highest Diversity score (12.0), although multimodality decreases (0.99).
- Adding the velocity loss (L_{vel}) alongside the distance loss (L_d) without adversarial loss leads to moderate performance, with a slightly better MAE (0.054) compared to adversarial loss alone, but higher FGD (46.4) and reduced diversity (8.00).
- The full configuration (L_{adv} + L_d + L_{vel}) slightly increases MAE (0.115) compared to the adversarial-distance combination and yields an FGD of 10.4, emphasizing feature-level realism at the expense of multimodality (0.73).

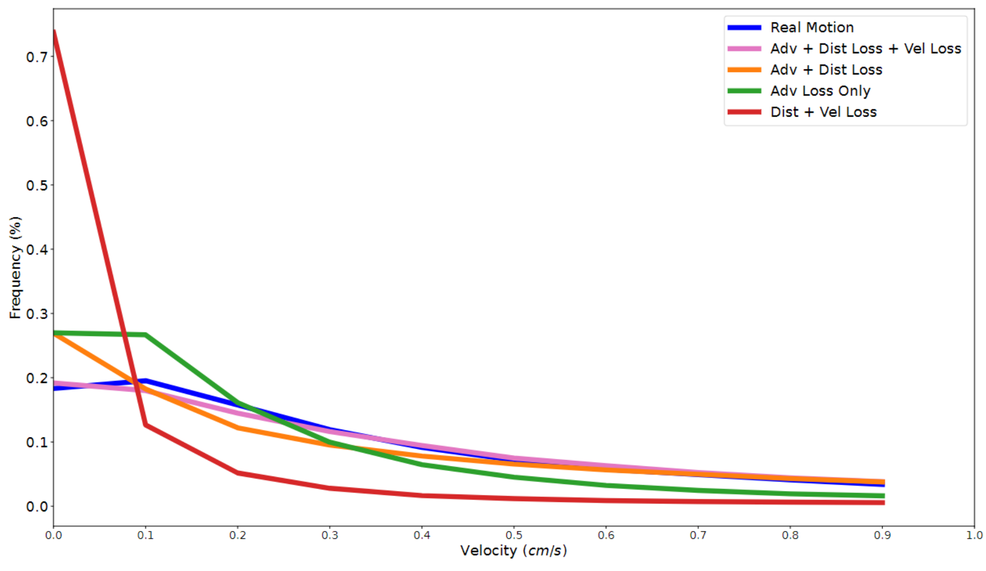


Figure 8 Velocity histograms of the joints for the ablation study with various loss functions.

Figure 8 illustrates the velocity histograms of joint motions generated under various loss function configurations. Among the tested conditions, the combination of adversarial loss, distance loss, and velocity loss (L_{adv} + L_d + L_{vel}) demonstrates superior performance in generating motion velocities closely resembling the real human motion profile. Specifically, while models trained solely with adversarial and distance losses (L_{adv} + L_d) result in lower spatial errors, their velocity profiles deviate from authentic human motion, exhibiting unnatural and abrupt transitions. Conversely, incorporating velocity loss into the training process effectively ensures that generated velocities align closely with real motion distributions. This close alignment is critical for producing smoother, perceptually natural gestures, enhancing realism and effectiveness in human-robot interaction contexts.

4.3 Baseline Comparison

To evaluate the performance of the proposed LSTM-GAN framework, a comparative analysis was conducted against CoSpeech [40]. The comparison leveraged quantitative metrics (Table III) and qualitative analysis (Figure 9) to assess the strengths and limitations of both approaches.

Quantitative Analysis

Table III
QUANTITATIVE COMPARISON WITH BASELINE MODELS TRAINED ON THE SAME DATASET

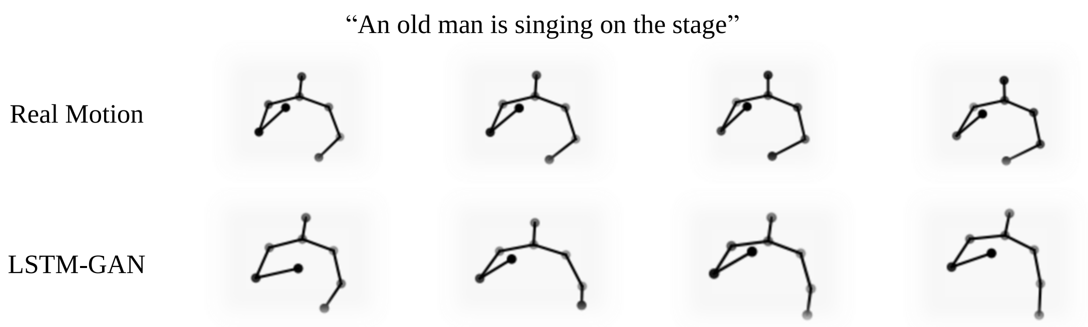
	Jerk →	**Acceleration →**	**Velocity →**	**MAE ↓**	**FGD ↓**	**Diversity ↑**	**MModality ↑**
Real Motion	5559 ± 147	304 ± 8.22	2.97 ± 2.37	-	-	-	-
LSTM-GAN	$\mathbf{5120 \pm 197}$	$\mathbf{280 \pm 10.6}$	$\mathbf{2.78 \pm 2.12}$	**0.115**	**10.4**	**11.5**	**0.73**
CoSpeech [40]	12278 ± 1442	671 ± 79.2	7.07 ± 4.67	0.354	273	21.2	21.4

As demonstrated in Table III, the LSTM-GAN framework exhibited superior performance in kinematic smoothness metrics compared to CoSpeech [40]:

- **Jerk:** The LSTM-GAN achieved $5120 \pm 197\ cm/s^3$, closely approximating the natural human motions benchmark ($5559 \pm 147\ cm/s^3$), while CoSpeech [40] generated motions with significantly higher jerk values ($12278 \pm 1442\ cm/s^3$), indicative of abrupt and unnatural movements.
- **Acceleration and Velocity:** The LSTM-GAN's acceleration ($280 \pm 10.6\ cm/s^2$) and velocity ($2.78 \pm 2.12\ cm/s$) profiles aligned closely with human motion characteristics. In contrast, CoSpeech [40] produced exaggerated dynamics ($671 \pm 79.2\ cm/s^2$ and $7.07 \pm 4.67\ cm/s$), deviating from natural kinematic patterns.

The LSTM-GAN also demonstrated higher pose accuracy, as evidenced by a lower mean absolute error (MAE = 0.115 vs. 0.354), and greater feature-level realism, with a Fréchet Gesture Distance (FGD) of 10.4 compared to CoSpeech's 273. These results indicate that the LSTM-GAN generates gestures that more closely resemble ground-truth human motions. However, CoSpeech [40] achieved higher diversity (21.2 vs. 11.5) and multimodality (21.4 vs. 0.73) scores, reflecting its capacity to produce a broader range of gesture variations.

Qualitative Analysis



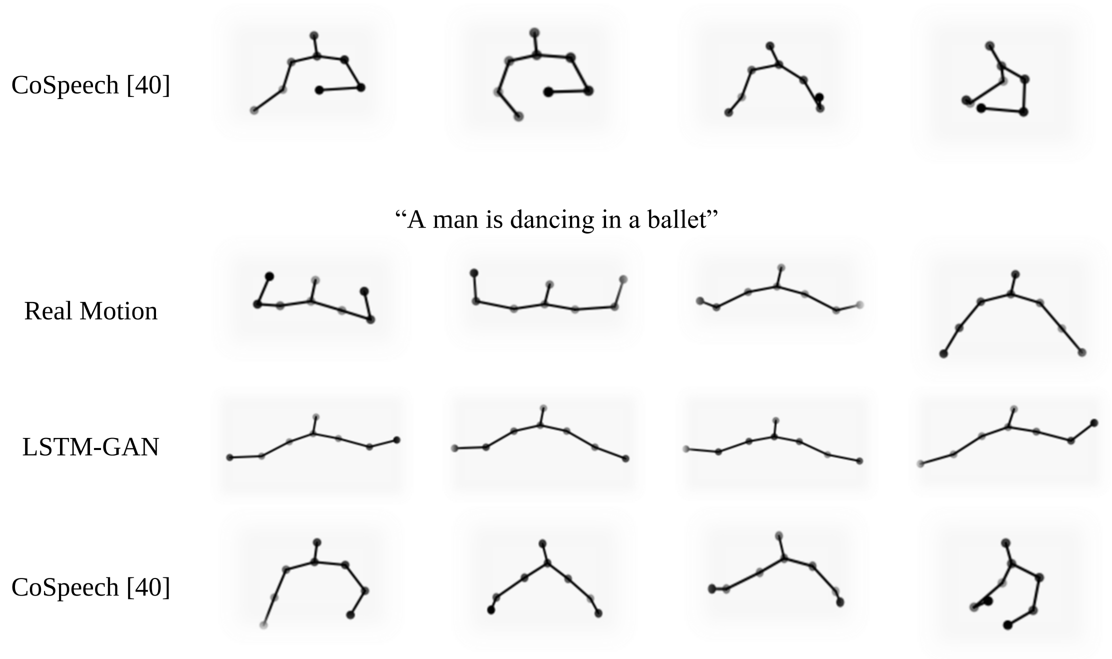
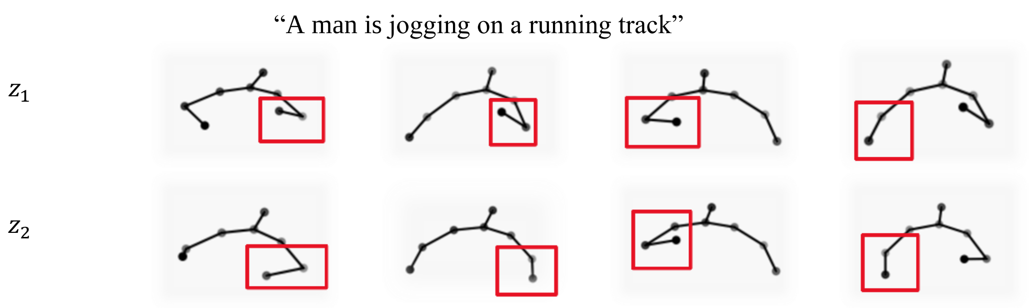


Figure 9 Qualitative comparison of generated motions for two prompts.

As illustrated in Figure 9, the LSTM-GAN framework generates gestures that exhibit smooth temporal transitions and strong alignment with textual semantics. For the prompt "An old man is singing on the stage", the synthesized motions reflect contextually appropriate upper-body gestures, such as rhythmic arm movements synchronized with implied speech cadence, demonstrating fluid coordination between gesture phases. Similarly, for "A man is dancing in a ballet", the model produces body motions with natural continuity, including synchronized limb movements that align with the artistic intent of the prompt. In contrast, CoSpeech generates motions that frequently diverge from textual meaning, such as generic arm swings for the "singing" prompt or disjointed upper-body movements for "ballet", failing to capture choreographic coherence.

Real motion benchmarks further emphasize the LSTM-GAN’s superiority: its outputs closely approximate human-like kinematic smoothness, with velocity profiles reflecting natural fluidity, while CoSpeech’s gestures display abrupt transitions and erratic velocity patterns. This qualitative comparison underscores the LSTM-GAN’s ability to integrate semantic grounding and temporal coherence, essential for contextually natural human-robot interaction.

4.4 Multimodal Gesture Generation



z_3

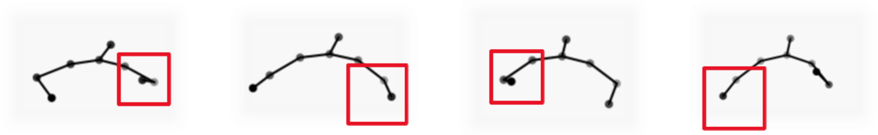


Figure 10 Multimodal gesture synthesis for the text 'A man is jogging on a running track' using varied latent noise vectors.

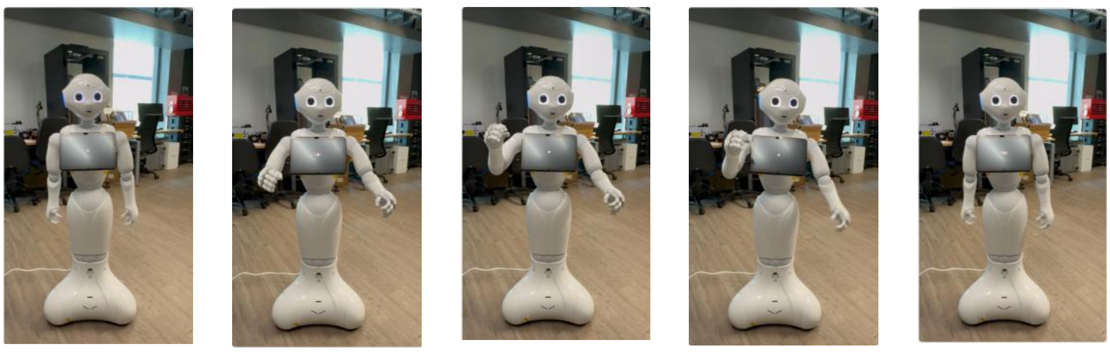
The LSTM-GAN framework enables multimodal gesture synthesis by sampling distinct latent vectors (z) from a Gaussian distribution ($\mathcal{N}(0,1)$), conditioned on the same textual input. As illustrated in Figure 10, the prompt "A man is jogging on a running track" generates three motion sequences from different latent vectors (z_1, z_2, z_3), each exhibiting stylistic variations while preserving the semantic core of jogging.

- z_1: Produces extended outward arm swings and pronounced stride dynamics, reflecting a high-energy jogging style.
- z_2: Yields tighter inward arm movements with a slightly hunched posture, indicative of a subdued or fatigued motion.
- z_3: Balances rhythmic limb coordination with compact gestures, demonstrating intermediate vigor.

These variations highlight the model's capacity to explore the latent space, generating diverse yet contextually consistent motions from a single text prompt. By modulating z, the framework captures inherent ambiguities in gesture interpretation—such as motion intensity and limb coordination—without compromising semantic alignment. This balance between diversity and realism is enforced through adversarial training, where the discriminator penalizes biomechanically implausible outputs, ensuring synthesized gestures remain within physiologically credible bounds. The results underscore the model's ability to address the rigidity of rule-based systems, offering adaptable gesture libraries tailored to nuanced interaction contexts.

4.5 Implementation on a Pepper Robot

"An old man is singing on the stage"



"A man is dancing in a ballet"

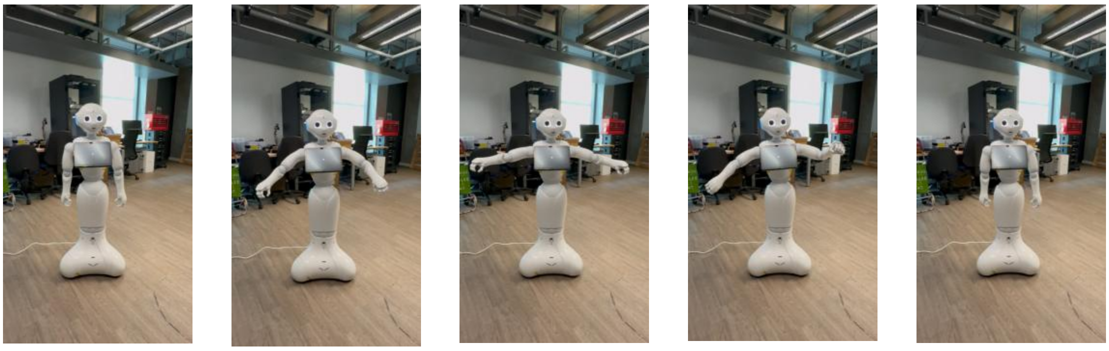
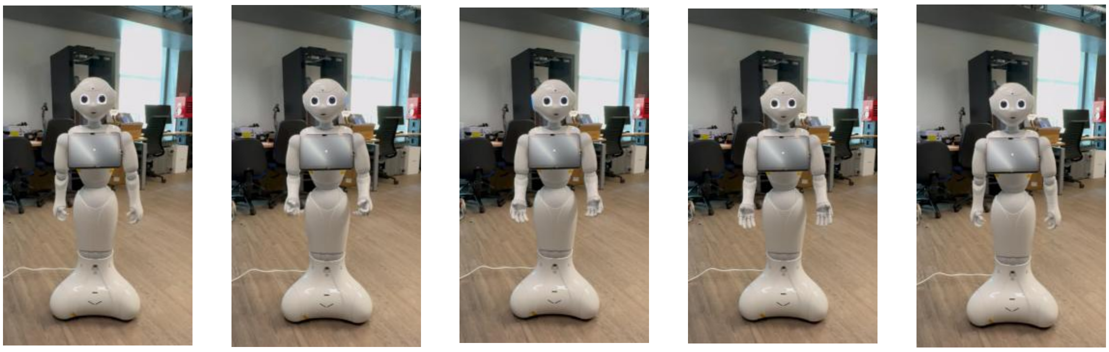


Figure 11 Generated motion sequences of a Pepper robot in real-world deployment scenarios, capturing two expressive scenarios: 'A man dancing ballet' and 'An old man singing on stage.'

“An old man is singing on the stage”



“A man is dancing in a ballet”

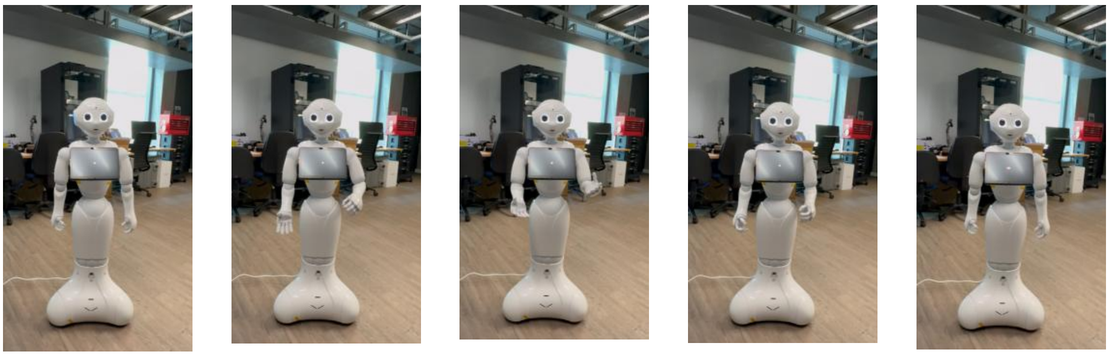


Figure 12 Generated motion sequences of a Pepper robot executing gestures via the NAOqi API ALAnimatedSpeech, demonstrating expressive scenarios: “A man dancing ballet” and “An old man singing on stage”.

The LSTM-GAN-generated motion sequences were transformed into executable joint angle trajectories and implemented on SoftBank’s Pepper robot for real-world human-robot interaction (HRI) tasks. This deployment demonstrated the practical applicability and context-aware adaptability of the proposed model.

Figures 11 and 12 visually illustrate two expressive scenarios: "An old man is singing on the stage" and "A man is dancing in a ballet." For each scenario, we present the gesture sequences generated by the proposed LSTM-GAN model (Figure 11) alongside gestures executed using Pepper’s built-in NAOqi API ALAnimatedSpeech (Figure 12), which employs a rule-based gesture generation approach.

As evident from Figure 11, gestures generated by the LSTM-GAN model dynamically adapt to the context and semantics of the provided textual input. In the scenario depicting singing, the robot performs rhythmic, coordinated arm movements that align naturally with the implied vocal performance. Similarly, for the ballet dancing scenario, gestures are characterized by graceful, synchronized upper-body movements that effectively mimic human ballet performance, clearly illustrating the model's contextual understanding and temporal coherence.

In contrast, Figure 12 highlights the limitations of NAOqi’s ALAnimatedSpeech, demonstrating gestures that lack alignment with specific textual contexts. The robot employs generic movements regardless of the nuanced differences implied by distinct textual prompts. For instance, gestures accompanying the ballet scenario do not exhibit the smoothness or choreographic alignment one would naturally expect. Similarly, gestures for the singing scenario appear overly generalized and fail to reflect the expressive intent inherent in the description.

This comparison underscores the strength of the proposed LSTM-GAN model in achieving context-aware, adaptive, and semantically coherent gesture generation, thus significantly enhancing the realism and perceived naturalness of interactions compared to the rigid, rule-based gestures produced by NAOqi’s ALAnimatedSpeech.

5 Project Management and Planning

5.1 Timeline and Milestone

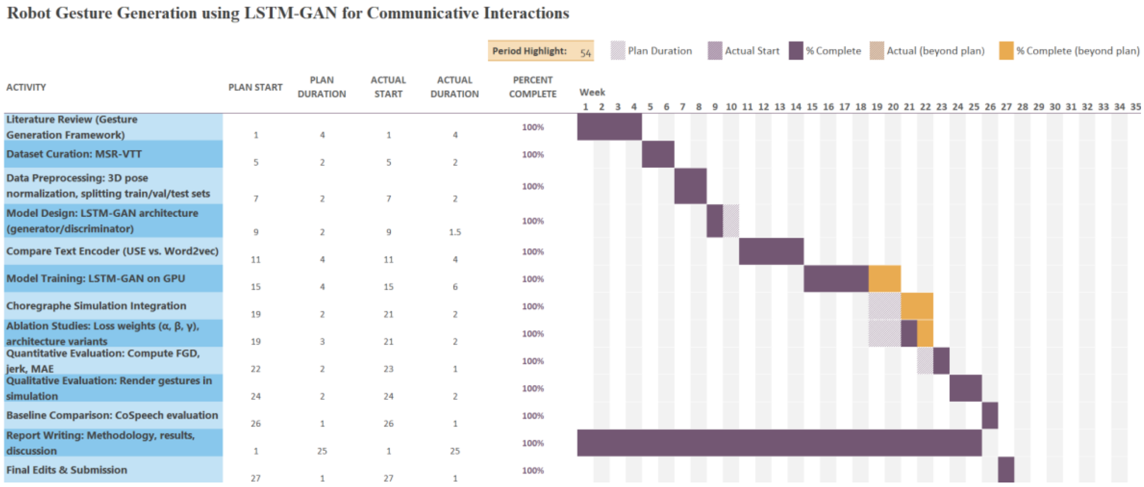


Figure 13 Planned vs Actual Gantt Chart for Project Progress

A detailed visualization of the project timeline, including planned versus actual progress, is provided in Figure 13. The project adhered to a structured 28-week timeline (9 October 2024 – 26 April 2025), with critical milestones aligned to the development of the LSTM-GAN framework for robot gesture generation.

The Literature Review phase commenced on 9 October 2024 and concluded as scheduled on 6 November 2024, establishing a foundation for subsequent technical work. Dataset curation and preprocessing followed, with the MSR-VTT dataset curated by 20 November 2024 and normalized by 4 December 2024, adhering strictly to the planned timeframe.

The Model Design phase, focused on defining the LSTM-GAN architecture, was completed ahead of schedule on 18 December 2024, demonstrating efficient architectural refinement. Implementation of the text encoder (comparing Word2Vec and Universal Sentence Encoder [35]) proceeded as planned, concluding punctually on 15 January 2025. Model Training encountered delays due to performance instability and hyperparameter tuning, extending its duration from 4 to 6 weeks. To recover from this delay, task overlaps, and weekend extensions were strategically utilized, concluding training by 26 February 2025.

Post-training phases, including Choregraphe simulation integration and ablation studies, were slightly adjusted yet maintained alignment with evaluation milestones. The final quantitative and qualitative analyses were completed by 16 April 2025. Regular monitoring and adaptive management strategies successfully maintained overall project timelines, ensuring final submission on 28 April 2025.

5.2 Resource Allocation

The project's resource allocation prioritized computational efficiency and methodological rigor. Hardware resources centred on an NVIDIA GeForce GTX 1080 Ti GPU, which facilitated the intensive training of the LSTM-GAN framework. The School of Electronics and Computer Science's High-Performance Computing (HPC) cluster supported parallelized hyperparameter tuning and batch processing, optimizing time-to-convergence during model training.

Dataset curation relied on the MSR-Video-to-Text (MSR-VTT) and KIT Motion-Language corpora, preprocessed for 3D joint coordinate extraction and text-motion alignment. Choregraphe 2.5.10.7 facilitated motion validation and deployment on the Pepper robot, ensuring kinematic feasibility prior to deployment. Motion visualization leveraged Blender for 3D rendering, while Python scripts standardized metric computations.

Human resources included weekly supervisory consultations for architectural refinements, with code integrity ensured through iterative testing against validation benchmarks and ablation studies. Third-party tools, such as TensorFlow, Universal Sentence Encoder [35], and the MSR-VTT dataset, were integrated with explicit citations in adherence to academic integrity standards. All experimental workflows and code implementations were developed independently.

5.3 Risk Management

The project's risk management strategy focused on pre-emptively identifying and mitigating challenges across technical, data-related, and operational domains to ensure adherence to timelines and deliverables. A critical technical risk stemmed from training instability in the LSTM-GAN framework, particularly oscillations caused by adversarial and kinematic loss balancing. To address this, spectral normalization was applied to the discriminator's convolutional layers to stabilize gradient flow, and iterative hyperparameter tuning was conducted using the validation set. The Adam optimizer's learning rate (0.00002) and loss weights ($\alpha = 1$, $\beta = 10$, $\gamma = 5$) were empirically calibrated to harmonize adversarial training with motion fidelity. Access to the High-Performance Computing (HPC) cluster ensured efficient resource allocation for parallelized experiments, circumventing computational bottlenecks.

Data-related risks centred on the limitations of the MSR-VTT dataset, specifically its lack of fine-grained annotations for directional or object-centric gestures (e.g., "left hand holding a microphone"). While preprocessing steps—including 3D pose estimation via Zhou et al.'s framework and joint vector normalization—enhanced motion consistency, the textual descriptions' emphasis on broad actions (e.g., "singing") restricted the model's ability to synthesize gestures with compositional specificity. This limitation was acknowledged during dataset curation, with compensatory reliance on the Universal Sentence Encoder [35] to maximize semantic alignment between text and motion.

Operational risks involved deploying synthesized gestures to the Pepper robot, as generated joint angles occasionally exceeded mechanical limits. To mitigate this, post-hoc normalization scaled joint trajectories to Pepper's kinematic constraints, and validation in Choregraphe ensured motion feasibility prior to real-world deployment. Blender-based 3D rendering further facilitated iterative refinements of gesture sequences.

Ethical risks were minimized by adhering to the University's guidelines, with no involvement of human or animal participants. Reproducibility was ensured through rigorous documentation of code, training workflows, and dataset preprocessing steps. By aligning risk mitigation with iterative validation and adaptive planning, the project maintained methodological integrity while addressing emergent challenges, achieving its objectives within the revised timeline.

6 Discussion

6.1 Strengths of the LSTM-GAN Framework

The LSTM-GAN framework effectively translates high-level textual intent into coherent gesture sequences, demonstrating strong alignment between input prompts and generated motions. For example, prompts like “A man is jogging on a running track” or “An old man is singing on the stage” reliably produce gestures that reflect the described actions, with smooth transitions and natural dynamics. Quantitative results, such as the model’s low Fréchet Gesture Distance (FGD = 10.4) and jerk values (5120 cm/s^3) comparable to real motion benchmarks, confirm its ability to synthesize realistic gestures. Furthermore, the adversarial training paradigm encourages diversity, enabling the model to generate varied motions for the same prompt, a critical feature for avoiding robotic repetitiveness in human-robot interactions.

6.2 Weaknesses and Limitations

Despite these successes, the model struggles to capture fine-grained details embedded in textual descriptions. For instance, when given prompts specifically directional or object-centric details - such as “An old man is singing on the stage with his left hand holding a microphone" versus “An old man is singing on the stage with his right hand holding a microphone" – the generated gestures default to generic singing motions (Figure 9), failing to distinguish between left- or right-handed actions. This limitation stems from two key factors:

1. **Text Encoder Constraints**
   The Universal Sentence Encoder (USE) [35], while robust for general semantic understanding, may overlook positional or compositional cues such as “left hand”, “holding a microphone”. USE embeddings prioritize overall sentence meaning over fine-grained descriptors, leading to ambiguity in motion synthesis. A potential solution involves integrating domain-specific encoders or attention mechanisms to highlight spatial or object-related keywords during text processing.

2. **Dataset Annotation Gaps**
   The training dataset lacks explicit annotations for directional or object-oriented details. Although it contains nearly 30000 text-motion pairs, the textual descriptions emphasize broad actions such as “singing” rather than compositional specifics. Without labelled examples distinguishing object interactions, the model cannot learn to associate such details with precise joint-angle adjustments.

7 Conclusion and Future Work

7.1 Conclusion

This project successfully developed and validated an LSTM-GAN framework designed to address critical limitations in robot gesture generation, specifically context insensitivity, temporal incoherence, and expressiveness gaps. By integrating Long Short-Term Memory networks for temporal modeling with Generative Adversarial Networks for realistic motion synthesis, the proposed model generates gestures that are contextually relevant and temporally coherent. Quantitative evaluations demonstrated significant improvements in gesture realism, pose accuracy, and kinematic smoothness compared to baseline models such as CoSpeech.

A practical deployment of the developed framework on the SoftBank Pepper robot illustrated clear advantages over conventional rule-based gesture generation approaches. Specifically, gestures synthesized by the proposed LSTM-GAN model dynamically adapt to textual contexts, effectively enhancing user interaction through more natural and expressive robot behaviours. In contrast, the NAOqi ALAnimatedSpeech approach exhibited limitations in semantic alignment, underscoring the model's superior context-aware capabilities.

Overall, this work bridges the gap between theoretical advancements in gesture generation algorithms and practical applicability in human-robot interaction scenarios, laying a foundation for future socially intelligent robotic systems capable of adaptive and meaningful communication.

7.2 Future Work

To further refine the model's capability to capture fine-grained textual details and improve gesture specificity, two focused extensions are proposed:

1. **Advanced Text Encoder via Large Language Models (LLMs):**
   Replace the Universal Sentence Encoder (USE) [35] with state-of-the-art LLMs such as BERT [18] or GPT [41] to better parse compositional and spatial nuances in textual prompts. LLMs, with their superior semantic understanding and attention mechanisms, can disambiguate directional cues (e.g., "left hand" vs "right hand") and object interactions (e.g., "holding a microphone"). This upgrade would enable the model to generate gestures that align precisely with fine-grained textual specifications.

2. **Integrations of the KIT Motion-Language Dataset:**
   Augment training with the KIT dataset [42], which pairs rich human motion sequences with detailed textual annotations, including object affordances and directional descriptors. KIT's granular annotations (e.g., "raising left arm to point at an object") would address the current dataset's lack of compositional labels, allowing the model to learn associations between textual specifics and joint-angle

adjustments. This would enhance the system's ability to synthesize gestures that reflect subtle intent, such as handedness or object manipulation.

By advancing these directions, future work will solidify the framework's role in enabling socially intelligent robots to operate seamlessly in unstructured, real-world environments.

References

[1] J. Leusmann, S. Villa, T. Liang, C. Wang, A. Schmidt, and S. Mayer, "An Approach to Elicit Human-Understandable Robot Expressions to Support Human-Robot Interaction," 2018. [Online]. Available: https://arxiv.org/html/2410.01071v1

[2] B. Heleen, M. Abubakar, and S. Ahmad, "The Role of LSTMs in Time Series and Text Processing," Apr. 2025, doi: 10.2139/ssrn.5206958.

[3] T. Boesen, "DeepMind's GenEM is revolutionizing robot expressive behaviors | Okoone," Apr. 2024. [Online]. Available: https://www.okoone.com/spark/technology-innovation/deepminds-genem-is-revolutionizing-robot-expressive-behaviors/

[4] S. Tang and C. Dondrup, "Gesture Generation from Trimodal Context for Humanoid Robots," Sep. 2024.

[5] S. Nyatsanga, T. Kucherenko, C. Ahuja, G. E. Henter, and M. Neff, "A Comprehensive Review of Data-Driven Co-Speech Gesture Generation," Jan. 2023, doi: 10.1111/cgf.14776.

[6] J. Cassell, Y. Nakano, T. Bickmore, C. L. Sidner, and C. Rich, "Non-verbal cues for discourse structure," Apr. 2001, doi: 10.3115/1073012.1073028.

[7] S. Kopp *et al.*, "Towards a Common Framework for Multimodal Generation: The Behavior Markup Language," 2006. [Online]. Available: https://www.ofai.at/~hannes.pirker/papers/kopp_etal_IVA_2006.pdf

[8] K. D. Neff and R. Vonk, "Self-Compassion Versus Global Self-Esteem: Two Different Ways of Relating to Oneself," *J Pers*, vol. 77, no. 1, pp. 23–50, 2009, doi: https://doi.org/10.1111/j.1467-6494.2008.00537.x.

[9] C.-M. Huang and B. Mutlu, "Learning-based modeling of multimodal behaviors for humanlike robots," *Human-Robot Interaction*, Apr. 2014, doi: 10.1145/2559636.2559668.

[10] K. D. Neff and C. K. Germer, "A Pilot Study and Randomized Controlled Trial of the Mindful Self-Compassion Program," *J Clin Psychol*, vol. 69, no. 1, pp. 28–44, 2013, doi: https://doi.org/10.1002/jclp.21923.

[11] B. Ravenet, C. Pelachaud, C. Clavel, and S. Marsella, "Automating the Production of Communicative Gestures in Embodied Characters," *Front Psychol*, vol. 9, Apr. 2018, doi: 10.3389/fpsyg.2018.01144.

[12] C. Ahuja, D. Lee, R. Ishii, and L.-P. Morency, "No Gestures Left Behind: Learning Relationships between Spoken Language and Freeform Gestures," 2020. [Online]. Available: https://aclanthology.org/2020.findings-emnlp.170.pdf

[13] Z. GHAHRAMANI, "AN INTRODUCTION TO HIDDEN MARKOV MODELS AND BAYESIAN NETWORKS," *Intern J Pattern Recognit Artif Intell*, vol. 15, pp. 9–42, Apr. 2001, doi: 10.1142/s0218001401000836.

[14] J. Qiu, L. Xu, J. Zhai, and L. Luo, "Extracting Causal Relations from Emergency Cases Based on Conditional Random Fields," *Procedia Comput Sci*, vol. 112, pp. 1623–1632, 2017, doi: https://doi.org/10.1016/j.procs.2017.08.252.

[15] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Comput*, vol. 9, pp. 1–42, Apr. 1997, doi: 10.1162/neco.1997.9.1.1.

[16] I. Goodfellow *et al.*, "Generative Adversarial Nets." [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2014/file/f033ed80deb0234979a61f95710dbe25-Paper.pdf

[17] Y. Yoon, "Youngwoo Yoon - Co-Speech Gesture Generation," 2017. [Online]. Available: https://sites.google.com/view/youngwoo-yoon/projects/co-speech-gesture-generation

[18] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," Apr. 2018. [Online]. Available: https://arxiv.org/abs/1810.04805

[19] F. Yunus, "Prediction of Gesture Timing and Study About Image Schema for Metaphoric Gestures," 2021. doi: 10.13140/RG.2.2.35346.12480.

[20] A. Vaswani *et al.*, "Attention Is All You Need," Apr. 2017. [Online]. Available: https://arxiv.org/pdf/1706.03762

[21] S. R. B. Archives and A. Kendon, "An Agenda for Gesture Studies," *The Semiotic Review of Books*, vol. 7, Apr. 1996.

[22] G. E. Henter, S. Alexanderson, and J. Beskow, "MoGlow: probabilistic and controllable motion synthesis using normalising flows," *ACM Trans. Graph.*, vol. 39, no. 6, Nov. 2020, doi: 10.1145/3414685.3417836.

[23] Y. Ferstl and R. McDonnell, “Investigating the use of recurrent motion modelling for speech gesture generation,” in *Proceedings of the 18th International Conference on Intelligent Virtual Agents*, in IVA ’18. New York, NY, USA: Association for Computing Machinery, 2018, pp. 93–98. doi: 10.1145/3267851.3267898.

[24] M. Loper, J. Romero, and M. Black, “SMPL: A Skinned Multi-Person Linear Model Naureen Mahmood †1 Gerard Pons-Moll †1,” 2015. [Online]. Available: https://files.is.tue.mpg.de/black/papers/SMPL2015.pdf

[25] T. Kucherenko, P. Jonell, Y. Yoon, P. Wolfert, and G. E. Henter, “A Large, Crowdsourced Evaluation of Gesture Generation Systems on Common Data: The GENEA Challenge 2020,” Apr. 2021, doi: 10.1145/3397481.3450692.

[26] S. Ghorbani, Y. Ferstl, D. Holden, N. F. Troje, and M.-A. Carbonneau, “ZeroEGGS: Zero-shot Example-based Gesture Generation from Speech,” 2022. [Online]. Available: https://arxiv.org/abs/2209.07556

[27] S. Alexanderson, R. Nagy, J. Beskow, and G. E. Henter, “Listen, Denoise, Action! Audio-Driven Motion Synthesis with Diffusion Models,” *ACM Trans. Graph.*, vol. 42, no. 4, Jul. 2023, doi: 10.1145/3592458.

[28] M. Li *et al.*, “Towards Visual Text Grounding of Multimodal Large Language Model,” 2025. [Online]. Available: https://arxiv.org/abs/2504.04974

[29] C. Guo *et al.*, “Generating Diverse and Natural 3D Human Motions from Text.” [Online]. Available: https://openaccess.thecvf.com//content/CVPR2022/papers/Guo_Generating_Diverse_and_Natural_3D_Human_Motions_From_Text_CVPR_2022_paper.pdf

[30] N. Yehia, “Understanding Long Short-Term Memory (LSTM) Networks,” Apr. 2024. [Online]. Available: https://mlarchive.com/deep-learning/understanding-long-short-term-memory-networks/

[31] M. Del Pra, “Generative Adversarial Networks,” Apr. 2023. [Online]. Available: https://medium.com/@marcodelpra/generative-adversarial-networks-dba10e1b4424

[32] J. Xu, T. Mei, T. Yao, and Y. Rui, “MSR-VTT: A Large Video Description Dataset for Bridging Video and Language,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 5288–5296. doi: 10.1109/CVPR.2016.571.

[33] S.-E. Wei, V. Ramakrishna, T. Kanade, and Y. Sheikh, “Convolutional Pose Machines,” Apr. 2016, doi: 10.48550/arXiv.1602.00134.

[34] C. Zhou, Cheng Fang, X. Wang, Z. Li, and N. Tsagarakis, “A generic optimization-based framework for reactive collision avoidance in bipedal locomotion,” *2016 IEEE International Conference on Automation Science and Engineering (CASE)*, pp. 1026–1033, Apr. 2016, doi: 10.1109/coase.2016.7743516.

[35] D. Cer *et al.*, “Universal Sentence Encoder,” Mar. 2018.

[36] T. Kucherenko *et al.*, “Gesticulator: A framework for semantically-aware speech-driven gesture generation,” *arXiv (Cornell University)*, Apr. 2020, doi: 10.1145/3382507.3418815.

[37] G. Sun, Y. Wong, Z. Cheng, M. S. Kankanhalli, W. Geng, and X. Li, “DeepDance: Music-to-Dance Motion Choreography With Adversarial Learning,” *IEEE Trans Multimedia*, vol. 23, pp. 497–509, 2021, doi: 10.1109/TMM.2020.2981989.

[38] antmaio, “GitHub - antmaio/FrechetMotionDistance: Fréchet Gesture Distance from (Yoon et al.) exploration and eventual improvment,” 2022. [Online]. Available: https://github.com/antmaio/FrechetMotionDistance

[39] J. Zhang *et al.*, “T2M-GPT: Generating Human Motion from Textual Descriptions with Discrete Representations,” 2023. [Online]. Available: https://arxiv.org/abs/2301.06052

[40] H. Ahn, T. Ha, Y. Choi, H. Yoo, and S. Oh, “Text2Action: Generative Adversarial Synthesis from Language to Action,” Oct. 2017.

[41] AWS, “What is GPT AI? - Generative Pre-Trained Transformers Explained - AWS,” 2024. [Online]. Available: https://aws.amazon.com/what-is/gpt/

[42] M. Plappert, C. Mandery, and T. Asfour, “KIT Motion-Language Dataset,” 2017. [Online]. Available: https://motion-annotation.humanoids.kit.edu/dataset/

Appendices A. Project Brief

Project Name	Robot Gesture Generation using LSTM-GAN for Communicative Interactions
Name	Liang Kai Jie
Project's Supervisor	Dr Tan Viet Tuyen Nguyen
Problem	Generating realistic robot gestures from textual descriptions is a complex task in natural language processing and motion synthesis for communicative robots. The challenge lies in understanding the semantic meaning of text and translating it into gestures that enhance robot-human communication. The project aims to enable a robot to interact conversationally with humans via a chatbot, while simultaneously performing communicative gestures that correspond to the conversation, improving natural engagement during interactions.
Goals	1. Develop an LSTM-GAN model capable of learning the complex mapping between textual inputs and corresponding robot gestures for communicative purpose. 2. Enable the robot to interact conversationally with humans through a chatbot interface, while performing appropriate communicative gestures based on the conversation content. 3. Evaluate the model's ability to generalize across different types of communicative text inputs and measure its performance in generating gestures that align with spoken text.
Scope	The scope of this project is to focus on generating robot gestures from short textual descriptions. This includes: 1. Designing the LSTM-based GAN architecture to handle sequential input (text) and output (communicative gesture sequences). 2. Training the model by utilizing an existing dataset that pairs textual descriptions with corresponding human action data. 3. Testing the model to ensure it captures the relationship between different action verbs, and gesture patterns appropriate for communication. 4. Evaluating the model's performance on various textual inputs to assess its ability to generate context-appropriate gestures.

Appendices B. Data Archive Structure

```
Archive Contents
├── **2/**
│   └── Universal Sentence Encoder files  # Pre-trained USE model files for text encoding
│
├── **Data/**
│   ├── pose/                    # Raw 3D pose sequences (folder)
│   ├── script/                  # Text scripts aligned with poses (folder)
│   ├── mean_pose.mat            # Mean pose (MATLAB data) for normalization
│   ├── metadata.npz             # Dataset metadata (e.g., labels, timestamps)
│   ├── text_script.npy          # Text embeddings for all sequences (preprocessed)
│   ├── total_script.txt         # Raw textual descriptions for all gestures
│   ├── train_action.npy         # Training motion sequences
│   ├── train_script.npy         # Text embeddings for training set
│   ├── val_action.npy           # Validation motion sequences
│   ├── val_script.npy           # Text embeddings for validation set
│   ├── test_action.npy          # Test motion sequences
│   ├── test_script.npy          # Text embeddings for test set
│
├── **Models/**
│   └── model_epoch_150.keras    # Final trained LSTM-GAN model (150 epochs)
│
├── **Notebooks/**
│   ├── eval_model.ipynb         # Model evaluation (MAE, jerk, velocity)
│   ├── evaluate_FGD.ipynb       # Fréchet Gesture Distance computation
│   ├── feat_extraction.ipynb    # Feature extraction from motion data
│   ├── MModality_generation.ipynb   # Multimodality score calculations
│   ├── static_frame.ipynb       # Static pose visualization
│   ├── test_GAN.ipynb           # GAN inference and testing
│   └── testing.ipynb            # General testing scripts
│
├── **Results/**                 # Best model outputs (plots)
│
├── **src/**
│   ├── Data/                    # Data loading and preprocessing scripts
│   ├── Evaluate/                # Evaluation metric implementations
│   ├── Pepper_Implementation/   # Pepper robot deployment code
│   ├── Train/                   # Training scripts for LSTM-GAN
│   ├── utils/                   # Helper functions (e.g., inverse kinematics)
│   ├── Validate/                # Validation workflows
│   └── Visualisation/           # Motion rendering and animation tools
│
├── environment.yml              # Conda environment dependencies
└── job.slurm                    # SLURM job script for HPC cluster training
```